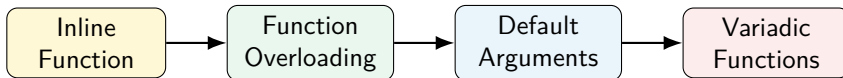


# Object Oriented Programming System

Course Code: 25CS301

## Lecture 4

Advanced Function Features in C++



B.Tech III Semester

Suggested Duration: 60 Minutes

# Lecture Roadmap

- ① Lecture Positioning
- ② Quick Recap of Functions
- ③ Inline Functions
- ④ Function Overloading
- ⑤ Default Arguments
- ⑥ Variadic Functions
- ⑦ Integrated Beginner Programs
- ⑧ Common Mistakes and Debugging
- ⑨ Exam and Viva Preparation
- ⑩ Lecture Summary

# Lecture Roadmap

- ① Lecture Positioning
- ② Quick Recap of Functions
- ③ Inline Functions
- ④ Function Overloading
- ⑤ Default Arguments
- ⑥ Variadic Functions
- ⑦ Integrated Beginner Programs
- ⑧ Common Mistakes and Debugging
- ⑨ Exam and Viva Preparation
- ⑩ Lecture Summary

# Where Lecture 4 Fits in Unit 1

## Previous Lecture

We studied ordinary functions, function declaration, function definition, function call, actual parameters, formal parameters, call by value, call by reference and call by address.

## Today's Lecture

We now improve ordinary functions using C++ features: inline functions, function overloading, default arguments and variadic functions.

## Why It Matters for OOP

These ideas appear again in constructors, overloaded constructors, overloaded operators, templates and generic programming.

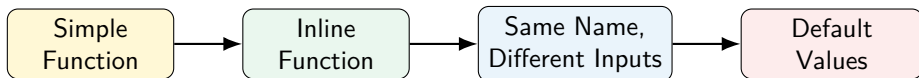
# Learning Objectives

By the end of this lecture, students will be able to:

- explain the need for inline functions;
- write simple inline functions correctly;
- overload functions using different parameter lists;
- identify valid and invalid function overloading;
- use default arguments in function calls;
- avoid ambiguity between overloading and default arguments;
- understand the basic idea of variadic functions;
- connect these concepts with constructors and operator overloading.

# Beginner-Friendly Progression

Do not start with a large program. Start with one function and one integer. Then slowly move to two functions, then three overloaded functions, then default values.



# Lecture Roadmap

- ① Lecture Positioning
- ② Quick Recap of Functions
- ③ Inline Functions
- ④ Function Overloading
- ⑤ Default Arguments
- ⑥ Variadic Functions
- ⑦ Integrated Beginner Programs
- ⑧ Common Mistakes and Debugging
- ⑨ Exam and Viva Preparation
- ⑩ Lecture Summary

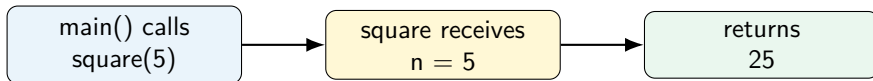
# A Very Simple Function

```
1 #include <iostream>
2 using namespace std;
3
4 int square(int n)
5 {
6     return n * n;
7 }
8
9 int main()
10 {
11     cout << square(5);
12     return 0;
13 }
```

A function receives input, performs a task and may return a result.



# Function Call Flow



Before teaching inline functions, make students revise what happens during a normal function call.

# Normal Function Call: Hidden Work

When a function is called, the program usually performs some extra work:

- control jumps from caller to function;
- parameters are prepared;
- function body executes;
- result returns to caller;
- control comes back to the next statement.

Students often think a function call is only a single line. Internally, it involves control transfer and some overhead.

# Lecture Roadmap

- ① Lecture Positioning
- ② Quick Recap of Functions
- ③ Inline Functions**
- ④ Function Overloading
- ⑤ Default Arguments
- ⑥ Variadic Functions
- ⑦ Integrated Beginner Programs
- ⑧ Common Mistakes and Debugging
- ⑨ Exam and Viva Preparation
- ⑩ Lecture Summary

# Need for Inline Functions

## Problem

Some functions are very small but are called many times.

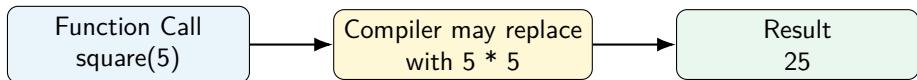
Examples: square of a number, maximum of two numbers, simple getter functions, small mathematical formulas.

## Idea

Instead of jumping to the function again and again, the compiler may replace the function call with the actual function body.

# Meaning of Inline Function

An inline function is a function for which the compiler is requested to expand the function body at the place of function call.



# Syntax of Inline Function

```
1 inline return_type function_name(parameter_list)
2 {
3     // small function body
4 }
```

The keyword `inline` is a request to the compiler, not an order.

Use inline functions only for small and frequently used functions.

# Program 1: Inline Square Function

```
1 #include <iostream>
2 using namespace std;
3
4 inline int square(int n)
5 {
6     return n * n;
7 }
8
9 int main()
10 {
11     cout << "Square = " << square(6);
12     return 0;
13 }
```

## Output

Square = 36

# Dry Run: Inline Square Function

| Statement               | Meaning                         |
|-------------------------|---------------------------------|
| <code>square(6)</code>  | Function is called with value 6 |
| <code>n = 6</code>      | Formal parameter receives 6     |
| <code>return n*n</code> | Returns 6*6                     |
| <b>Result</b>           | 36                              |

Do not say that inline always improves speed. That is false. It may improve speed for tiny functions, but it can also increase code size.



## Program 2: Inline Maximum Function

```
1 #include <iostream>
2 using namespace std;
3
4 inline int maximum(int a, int b)
5 {
6     return (a > b) ? a : b;
7 }
8
9 int main()
10 {
11     cout << "Maximum = " << maximum(12, 8);
12     return 0;
13 }
```

### Output

Maximum = 12

# When Inline is Suitable

- Function body is very small.
- Function is called frequently.
- Function logic is simple.
- Function does not contain complex control flow.

Good examples: `square()`, `cube()`, `maximum()`, `getAge()`, `getMarks()`.

# When Inline is Not Suitable

Avoid inline functions when the function contains:

- long body;
- loops;
- recursion;
- switch cases;
- complex logic;
- static variables in problematic cases.

Writing `inline` before every function is bad programming. It makes the intention unclear and may increase code size.

# Inline Function vs Macro

| Macro                                   | Inline Function                  |
|-----------------------------------------|----------------------------------|
| Handled by preprocessor                 | Handled by compiler              |
| No type checking                        | Type checking available          |
| Can produce unexpected results          | Safer than macro                 |
| Uses <code>#define</code>               | Uses <code>inline</code> keyword |
| Not recommended for function-like logic | Recommended for small functions  |

# Lecture Roadmap

- ① Lecture Positioning
- ② Quick Recap of Functions
- ③ Inline Functions
- ④ **Function Overloading**
- ⑤ Default Arguments
- ⑥ Variadic Functions
- ⑦ Integrated Beginner Programs
- ⑧ Common Mistakes and Debugging
- ⑨ Exam and Viva Preparation
- ⑩ Lecture Summary

# Need for Function Overloading

Suppose we need to add:

- two integers;
- three integers;
- two floating-point numbers.

Without overloading, we may write different names:

- `addInt()`
- `addThreeInt()`
- `addFloat()`

Function overloading allows the same function name to perform similar tasks with different parameter lists.

# Definition of Function Overloading

Function overloading is a compile-time polymorphism technique in which multiple functions have the same name but different parameter lists.

Parameter list may differ by:

- number of parameters;
- type of parameters;
- order of parameters.

## Program 3: Overloading by Number of Parameters

```
1 #include <iostream>
2 using namespace std;
3
4 int add(int a, int b)
5 {
6     return a + b;
7 }
8
9 int add(int a, int b, int c)
10 {
11     return a + b + c;
12 }
13
14 int main()
15 {
16     cout << add(10, 20) << endl;
17     cout << add(10, 20, 30);
18     return 0;
19 }
```



# Output and Explanation

## Output

30

60

| Function Call              | Compiler Selection                   |
|----------------------------|--------------------------------------|
| <code>add(10,20)</code>    | Calls function with two parameters   |
| <code>add(10,20,30)</code> | Calls function with three parameters |

## Program 4: Overloading by Type of Parameters

```
1 #include <iostream>
2 using namespace std;
3
4 int add(int a, int b)
5 {
6     return a + b;
7 }
8
9 double add(double a, double b)
10 {
11     return a + b;
12 }
13
14 int main()
15 {
16     cout << add(5, 7) << endl;
17     cout << add(2.5, 3.4);
18     return 0;
19 }
```

# Output and Explanation

## Output

12

5.9

The compiler decides which overloaded function to call by matching the arguments given in the function call.

## Program 5: Overloading by Order of Parameters

```
1 #include <iostream>
2 using namespace std;
3
4 void show(int age, string name)
5 {
6     cout << name << " is " << age << " years old" << endl;
7 }
8
9 void show(string name, int age)
10 {
11     cout << name << " is " << age << " years old" << endl;
12 }
13
14 int main()
15 {
16     show(18, "Rahul");
17     show("Aman", 19);
18     return 0;
19 }
```

# Valid Function Overloading

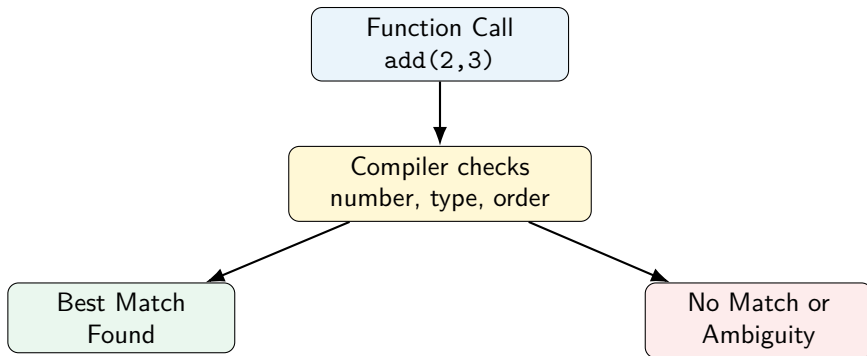
| Allowed                       |     | Reason                         |
|-------------------------------|-----|--------------------------------|
| <code>area(int)</code>        | and | Different number of parameters |
| <code>area(int,int)</code>    |     |                                |
| <code>display(int)</code>     | and | Different parameter type       |
| <code>display(double)</code>  |     |                                |
| <code>show(int,string)</code> | and | Different order of parameters  |
| <code>show(string,int)</code> |     |                                |

# Invalid Overloading: Return Type Alone

```
1 int display(int x)
2 {
3     return x;
4 }
5
6 double display(int x)    // Error
7 {
8     return x;
9 }
```

Functions cannot be overloaded only by changing the return type. The parameter list must be different.

# How Compiler Resolves Overloaded Calls



# Function Overloading and OOP

## Connection with Future Lectures

Function overloading prepares students for:

- constructor overloading;
- operator overloading;
- compile-time polymorphism;
- templates;
- generic programming.

Make this connection explicitly. Otherwise students treat overloading as an isolated topic.



# Lecture Roadmap

- ① Lecture Positioning
- ② Quick Recap of Functions
- ③ Inline Functions
- ④ Function Overloading
- ⑤ **Default Arguments**
- ⑥ Variadic Functions
- ⑦ Integrated Beginner Programs
- ⑧ Common Mistakes and Debugging
- ⑨ Exam and Viva Preparation
- ⑩ Lecture Summary

# Need for Default Arguments

Sometimes a function usually uses the same value, but occasionally we want to change it.

Example: calculating simple interest. If rate is usually 10 percent, we can make it a default argument.

Default arguments reduce the need to pass repeated common values.

# Syntax of Default Arguments

```
1 return_type function_name(type parameter = default_value)
2 {
3     // function body
4 }
```

If the caller does not provide that argument, the default value is used.

## Program 6: Greeting with Default Argument

```
1 #include <iostream>
2 using namespace std;
3
4 void greet(string name = "Student")
5 {
6     cout << "Hello, " << name << endl;
7 }
8
9 int main()
10 {
11     greet();
12     greet("Rahul");
13     return 0;
14 }
```

### Output

```
Hello, Student
Hello, Rahul
```

## Program 7: Simple Interest with Default Rate

```
1 #include <iostream>
2 using namespace std;
3
4 double simpleInterest(double p, double t, double r = 10.0)
5 {
6     return (p * r * t) / 100;
7 }
8
9 int main()
10 {
11     cout << simpleInterest(5000, 2) << endl;
12     cout << simpleInterest(5000, 2, 12.5);
13     return 0;
14 }
```

# Dry Run: Simple Interest

| Function Call                            | Meaning                |
|------------------------------------------|------------------------|
| <code>simpleInterest(5000,2)</code>      | Uses default rate 10.0 |
| <code>simpleInterest(5000,2,12.5)</code> | Uses given rate 12.5   |

## Outputs

First call: 1000

Second call: 1250

# Rule: Defaults Must Be Rightmost

```
1 void display(int a, int b = 10, int c = 20); // Correct
2
3 void display(int a = 5, int b, int c);      // Error
```

Once a parameter has a default value, all parameters to its right must also have default values.

# Default Arguments in Declaration

```
1 #include <iostream>
2 using namespace std;
3
4 void printLine(char ch = '-', int count = 10);
5
6 int main()
7 {
8     printLine();
9     printLine('*', 5);
10    return 0;
11 }
12
13 void printLine(char ch, int count)
14 {
15     for(int i = 1; i <= count; i++)
16         cout << ch;
17     cout << endl;
18 }
```



# Overloading vs Default Arguments

| Function Overloading                    | Default Arguments                        |
|-----------------------------------------|------------------------------------------|
| Multiple functions have same name       | Single function may serve multiple calls |
| Parameter lists differ                  | Some parameters have default values      |
| Compiler chooses best matching function | Missing values are filled automatically  |
| Useful for different logic              | Useful for common default values         |

# Danger: Ambiguity

```
1 void show(int a)
2 {
3     cout << a;
4 }
5
6 void show(int a, int b = 10)
7 {
8     cout << a + b;
9 }
10
11 int main()
12 {
13     show(5);    // Ambiguous
14 }
```

Do not combine overloading and default arguments carelessly. It can confuse the compiler.

# Lecture Roadmap

- ① Lecture Positioning
- ② Quick Recap of Functions
- ③ Inline Functions
- ④ Function Overloading
- ⑤ Default Arguments
- ⑥ Variadic Functions**
- ⑦ Integrated Beginner Programs
- ⑧ Common Mistakes and Debugging
- ⑨ Exam and Viva Preparation
- ⑩ Lecture Summary

# Need for Variadic Functions

Sometimes we do not know in advance how many arguments a function will receive.

Examples: printing many values, calculating sum of many numbers, logging messages, formatting output.

A variadic function can accept a variable number of arguments.

# Two Approaches in C++

| C-style Variadic Function                  | Modern C++ Variadic Template |
|--------------------------------------------|------------------------------|
| Uses <code>&lt;stdarg.h&gt;</code>         | Uses templates               |
| Less type-safe                             | More type-safe               |
| Beginner-level introduction possible       | Better for advanced C++      |
| Seen in functions like <code>printf</code> | Used in generic programming  |

For Lecture 4, teach only the idea and one simple example. Do not go deep into templates yet.

## Program 8: C-Style Variadic Sum

```
1 #include <iostream>
2 #include <cstdarg>
3 using namespace std;
4
5 int sum(int count, ...)
6 {
7     va_list args;
8     va_start(args, count);
9
10    int total = 0;
11    for(int i = 0; i < count; i++)
12        total += va_arg(args, int);
13
14    va_end(args);
15    return total;
16 }
17
18 int main()
19 {
20     cout << sum(3, 10, 20, 30);
21     return 0;
```

# Explanation of Variadic Sum

| Part                  | Meaning                                |
|-----------------------|----------------------------------------|
| <code>count</code>    | Number of values passed after it       |
| <code>...</code>      | Indicates variable number of arguments |
| <code>va_list</code>  | Stores argument list information       |
| <code>va_start</code> | Starts reading arguments               |
| <code>va_arg</code>   | Reads next argument                    |
| <code>va_end</code>   | Ends argument processing               |

# Important Warning on Variadic Functions

C-style variadic functions are not type-safe. If wrong argument types are passed, the compiler may not catch the mistake properly.

Use variadic functions only when needed. For normal beginner-level problems, arrays, vectors and function overloading are usually better.



# Modern Preview: Variadic Template

```
1 #include <iostream>
2 using namespace std;
3
4 void print()
5 {
6     cout << endl;
7 }
8
9 template<typename T, typename... Args>
10 void print(T first, Args... rest)
11 {
12     cout << first << " ";
13     print(rest...);
14 }
15
16 int main()
17 {
18     print(10, 20.5, "C++");
19     return 0;
20 }
```

# Lecture Roadmap

- ① Lecture Positioning
- ② Quick Recap of Functions
- ③ Inline Functions
- ④ Function Overloading
- ⑤ Default Arguments
- ⑥ Variadic Functions
- ⑦ Integrated Beginner Programs
- ⑧ Common Mistakes and Debugging
- ⑨ Exam and Viva Preparation
- ⑩ Lecture Summary

## Program 9: Area Calculator Using Overloading

```
1 #include <iostream>
2 using namespace std;
3
4 int area(int side)
5 {
6     return side * side;
7 }
8
9 int area(int length, int breadth)
10 {
11     return length * breadth;
12 }
13
14 int main()
15 {
16     cout << "Area of square = " << area(5) << endl;
17     cout << "Area of rectangle = " << area(6, 4);
18     return 0;
19 }
```

## Program 10: Simple Display Using Defaults

```
1 #include <iostream>
2 using namespace std;
3
4 void displayStudent(string name, int age = 18)
5 {
6     cout << "Name: " << name << endl;
7     cout << "Age : " << age << endl;
8 }
9
10 int main()
11 {
12     displayStudent("Rahul");
13     displayStudent("Aman", 20);
14     return 0;
15 }
```

This is simple enough for beginners and connects naturally to the Student class later.

# Program 11: Overloaded Display Functions

```
1 #include <iostream>
2 using namespace std;
3
4 void display(int value)
5 {
6     cout << "Integer: " << value << endl;
7 }
8
9 void display(double value)
10 {
11     cout << "Double: " << value << endl;
12 }
13
14 void display(string value)
15 {
16     cout << "String: " << value << endl;
17 }
18
19 int main()
20 {
21     display(10);
```

# Teaching Sequence for These Programs

- 1 First run the program.
- 2 Ask students to predict the output.
- 3 Ask which function will be called.
- 4 Draw the function call matching table.
- 5 Change one argument and ask what happens.
- 6 Finally discuss errors and ambiguity.

Students learn overloading best when they are asked to predict the function selected by the compiler.

# Lecture Roadmap

- ① Lecture Positioning
- ② Quick Recap of Functions
- ③ Inline Functions
- ④ Function Overloading
- ⑤ Default Arguments
- ⑥ Variadic Functions
- ⑦ Integrated Beginner Programs
- ⑧ Common Mistakes and Debugging**
- ⑨ Exam and Viva Preparation
- ⑩ Lecture Summary

# Common Mistake 1: Return Type Overloading

Changing only return type is not function overloading.

## Wrong Idea

`int show(int)` and `double show(int)` are different because return types are different.

## Correct Idea

The compiler chooses overloaded functions mainly from the arguments, not from the return type alone.



## Common Mistake 2: Wrong Default Argument Order

Default arguments must start from the right side.

| Correct                                                            | Wrong                                                              |
|--------------------------------------------------------------------|--------------------------------------------------------------------|
| <pre>void f(int a, int b=5) void g(int a, int b=5, int c=10)</pre> | <pre>void f(int a=5, int b) void g(int a=5, int b, int c=10)</pre> |

## Common Mistake 3: Too Much Inline

Do not mark every function as inline. It does not automatically make every program faster.

### Reason

Large inline functions can increase code size. Modern compilers may also ignore the inline request.

# Debugging Checklist

Before blaming the compiler, check:

- Is the function name spelled correctly?
- Are the number of arguments correct?
- Are argument types correct?
- Are default arguments written only once?
- Are default arguments placed from the right side?
- Is there ambiguity between overloaded functions?
- Is the function declared before use?

# Lecture Roadmap

- ① Lecture Positioning
- ② Quick Recap of Functions
- ③ Inline Functions
- ④ Function Overloading
- ⑤ Default Arguments
- ⑥ Variadic Functions
- ⑦ Integrated Beginner Programs
- ⑧ Common Mistakes and Debugging
- ⑨ Exam and Viva Preparation**
- ⑩ Lecture Summary

# Important Theory Questions

- 1 Define inline function. Explain with syntax and example.
- 2 What are the advantages and disadvantages of inline functions?
- 3 Define function overloading. Explain valid overloading cases.
- 4 Why can functions not be overloaded by return type alone?
- 5 Explain default arguments with example.
- 6 What is ambiguity in function overloading?
- 7 What is a variadic function? Explain with a simple example.

# Viva Questions

- 1 What is the purpose of `inline` keyword?
- 2 Is inline compulsory for compiler? Why?
- 3 What is compile-time polymorphism?
- 4 What is function signature?
- 5 Can return type alone overload a function?
- 6 What is the rule for default arguments?
- 7 Why are C-style variadic functions risky?
- 8 Where will overloading appear again in OOP?

# MCQs: Set 1

- ❶ Inline function is mainly used to reduce:  
A. Memory allocation    B. Function call overhead    C. Syntax errors    D. Header files
- ❷ Function overloading depends on:  
A. Return type only    B. Function name only    C. Parameter list    D. Comments
- ❸ Default arguments must be specified from:  
A. Left to right    B. Right to left    C. Middle    D. Anywhere

## Answers

1-B, 2-C, 3-B

## MCQs: Set 2

- ❶ Which is invalid overloading?
  - A. `f(int)` and `f(double)`
  - B. `f(int,int)` and `f(int)`
  - C. `int f(int)` and `double f(int)`
  - D. `f(int,string)` and `f(string,int)`
- ❷ C-style variadic functions use which header?
  - A. `<iostream>`
  - B. `<cmath>`
  - C. `<cstdarg>`
  - D. `<vector>`

### Answers

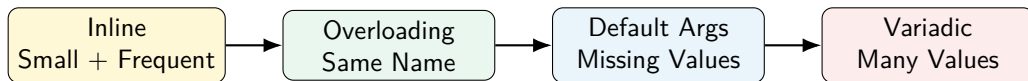
1-C, 2-C



# Lecture Roadmap

- ① Lecture Positioning
- ② Quick Recap of Functions
- ③ Inline Functions
- ④ Function Overloading
- ⑤ Default Arguments
- ⑥ Variadic Functions
- ⑦ Integrated Beginner Programs
- ⑧ Common Mistakes and Debugging
- ⑨ Exam and Viva Preparation
- ⑩ Lecture Summary**

# One-Slide Summary



Lecture 4 strengthens function design. These concepts are the foundation for constructor overloading, operator overloading and generic programming.

# Assignment

- 1 Write an inline function to calculate cube of a number.
- 2 Overload `area()` for circle, square and rectangle.
- 3 Write a function `bill()` with default tax rate.
- 4 Write overloaded `display()` functions for integer, double and string.
- 5 Write a short note on why return type alone cannot overload a function.

# Next Lecture Preview

## Lecture 5

### Modern C++ Basics: auto Keyword and Range-Based for Loop

- Why auto was introduced
- Type inference
- Range-based loop syntax
- Traversing arrays and vectors
- Link with STL and generic programming

# Thank You

Questions?

**Remember:** Good function design leads to good class design.